

Haskell.py Teacher's Edition

A Complete Course in Idiomatic Functional Python

nine chapters, 41 homework problems, 10 compare-and-contrast apps

J. L. Clements III

September 18, 2026

How to Take This Course

This book teaches `haskell.py` – `prelude.py`’s production sibling. Where `prelude.py` reimplements every tool from scratch for pedagogy, `haskell.py` WRAPS the standard library wherever it already has the tool (`scanl` is `itertools.accumulate`, `memo` is `functools.cache`) – one vocabulary, zero reimplementation. Failure is a value throughout: a true NOTHING sentinel keeps `None` legal as everyday data, parsers fail with `FAIL`, and `Either` carries `WHY`.

Each chapter has two movements. **Concepts**: every function introduced with its purpose, its worked doctest, and its definition. **Homework**: drills cement each tool, applies combine them, and challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in; `python3 -m doctest` is the referee.

Contents

How to Take This Course	iii
I The Nine Chapters	1
1 Point-Free Foundations	3
1.1 Concepts	3
1.2 Homework	5
2 Folds, Scans and Unfolds	7
2.1 Concepts	7
2.2 Homework	10
3 The List Gaps	13
3.1 Concepts	13
3.2 Homework	20
4 Maybe, Done Right	23
4.1 Concepts	23
4.2 Homework	24
5 Either: Failure With a Why	27
5.1 Concepts	27
5.2 Homework	28
6 do-Notation	31
6.1 Concepts	31
6.2 Homework	31
7 Parser Combinators	35
7.1 Concepts	35
7.2 Homework	37
8 Monoids at Work	39
8.1 Concepts	39
8.2 Homework	40
9 Capstone Labs	43
9.1 Homework	43

II	Compare and Contrast: Imperative vs Functional	45
	About this part	47
	10 calculator	49
	11 gradebook	51
	12 layers	53
	13 leaderboard	55
	14 logtrriage	57
	15 orderbatch	59
	16 orgchart	61
	17 payoff	63
	18 sensoralert	65
	19 wordfreq	67
III	Appendices	69
	A haskell.py, Complete	71

Part I

The Nine Chapters

Chapter 1

Point-Free Foundations

1.1 Concepts

1.1.1 identity

```
| identity = lambda x: x
```

1.1.2 const

```
| const    = lambda x: lambda _: x
```

1.1.3 flip

```
| flip     = lambda f: lambda a, b: f(b, a)
```

1.1.4 curry

```
| curry    = lambda f: lambda x: lambda y: f(x, y)
```

1.1.5 uncurry

```
| uncurry  = lambda f: lambda p: f(*p)
```

1.1.6 compose

Right-to-left composition, iterative (constant stack).

```
>>> compose(str, abs)(-3)
'3'
>>> o(str, abs)(-3)
'3'

def compose(*fs):
    def g(x):
        for f in reversed(fs):
            x = f(x)
        return x
    return g
```

1.1.7 o

```
| o = compose
```

1.1.8 pipe

Left-to-right application: `pipe(x, f, g) == g(f(x))`.

```
| >>> pipe(-3, abs, str)
| '3'
```

```
| def pipe(x, *fs):
|     for f in fs:
|         x = f(x)
|     return x
```

1.1.9 on

`Data.Function`: combine two arguments via a key.

```
| >>> on(sub, len)('haskell', 'py')
| 5
```

```
| def on(f, g):
|     return lambda x, y: f(g(x), g(y))
```

1.1.10 fst

```
| fst = lambda p: p[0]
```

1.1.11 snd

```
| snd = lambda p: p[1]
```

1.1.12 swap

```
| swap = o(tuple, reversed)
```

1.1.13 NOTHING

```
| NOTHING = _Sentinel("NOTHING")
```

1.1.14 FAIL

```
| FAIL = _Sentinel("FAIL")
```

1.1.15 _MISS

```
| _MISS = _Sentinel("_MISS")
```

1.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

1.1 Mirror a pair

drill

Write `mirror(p)`: the pair with its elements exchanged, point-free. Applying it twice must give the original back.

```
mirror(p) -> pair
|>>> mirror((1, 2))
| (2, 1)
|>>> mirror(mirror((3, 4)))
| (3, 4)
```

Solution.

```
| mirror = o(tuple, reversed)
```

Note. Reads as a sentence: tuple of reversed. No argument ever named.

1.2 Record projections

drill

A leaderboard record is a (name, score) pair. Write `name_of` and `score_of` point-free, without lambdas.

```
name_of(r) -> value; score_of(r) -> value
|>>> name_of(('ada', 95))
| 'ada'
|>>> score_of(('ada', 95))
| 95
```

Solution.

```
| name_of, score_of = itemgetter(0), itemgetter(1)
```

Note. `operator.itemgetter` is Python's fst/snd – and it slots straight into `map` and `sorted(key=...)`.

1.3 Default when missing

drill

Write `port_of(cfg)`: the 'port' entry of a config dict, defaulting to 8080 only when the key is absent. A key holding `None` is real data and must survive.

```
port_of(cfg) -> value
|>>> port_of({'port': 9000})
| 9000
|>>> port_of({})
| 8080
|>>> port_of({'port': None}) is None
| True
```

Solution.

```
| port_of = lambda cfg: fromMaybe(8080, maybe_get(cfg, 'port'))
```

Note. The prelude course's `or__default` cannot pass the third test: with `None-as-Nothing`, `None` IS the miss. The `NOTHING` sentinel is the fix.

1.4 Clamp to a range

applied

Write `clamp(lo, hi)`: a FUNCTION that forces a number into `[lo, hi]`. Build it from `max` and `min` with partial application – no `lambda` in the returned function.

```
clamp(lo, hi) -> (x -> value)
```

```
>>> clamp(0, 10)(15)
10
>>> clamp(0, 10)(-5)
0
>>> clamp(0, 10)(7)
7
```

Solution.

```
| clamp = lambda lo, hi: o(partial(max, lo), partial(min, hi))
```

Note. Right to left: cap above with `min(hi, .)`, then floor with `max(lo, .)`. Sections via `partial`.

1.5 Twice

drill

Write `twice(f)`: the function that applies `f` two times.

`twice(f) -> (x -> value)`

```
>>> twice(lambda n: n * 3)(2)
18
>>> twice(str.strip)(' hi ')
'hi'
```

Solution.

```
| twice = lambda f: o(f, f)
```

Note. `twice = o` on the diagonal. `twice(twice(f))` is `f` four times – combinators compose with themselves.

Chapter 2

Folds, Scans and Unfolds

2.1 Concepts

2.1.1 div

```
| div      = lambda a, b: a // b
```

2.1.2 mod

```
| mod      = lambda a, b: a % b
```

2.1.3 halve

```
| halve    = lambda n: n // 2
```

2.1.4 even

```
| even     = lambda n: n % 2 == 0
```

2.1.5 odd

```
| odd      = lambda n: n % 2 == 1
```

2.1.6 otherwise

```
| otherwise = True
```

2.1.7 quot

Truncating division, Haskell quot; rem's sign follows a.

```
| >>> quotRem(-7, 2)  
| (-3, -1)
```

```
| def quot(a, b):  
|     return -(-a // b) if (a < 0) != (b < 0) else a // b
```

2.1.8 rem

```
| rem      = lambda a, b: a - b * quot(a, b)
```

2.1.9 quotRem

```
| quotRem = lambda a, b: (quot(a, b), rem(a, b))
```

2.1.10 signum

```
| signum  = lambda x: (x > 0) - (x < 0)
```

2.1.11 succ

```
| succ    = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1
```

2.1.12 pred

```
| pred    = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1
```

2.1.13 foldl

foldl; seeds with the first element if no base is given.

```
| >>> foldl(lambda a, b: a - b, [1, 2, 3], 10)
4

| def foldl(f, xs, base=_MISS):
|     return _reduce(f, xs) if base is _MISS else _reduce(f, xs, base)
```

2.1.14 foldr

Right fold without recursion.

```
| >>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
| [1, 2, 3]

| def foldr(f, xs, base):
|     return _reduce(flip(f), reversed(list(xs)), base)
```

2.1.15 foldl1

```
| foldl1 = foldl
```

2.1.16 foldr1

```
| foldr1 = lambda f, xs: _reduce(flip(f), reversed(list(xs)))
```

2.1.17 scanl

All intermediate foldl states (itertools.accumulate).

```
| >>> scanl(add, [1, 2, 3], 0)
| [0, 1, 3, 6]
```

```
def scanl(f, xs, base=_MISS):
    return list(_accumulate(xs, f) if base is _MISS
                else _accumulate(xs, f, initial=base))
```

2.1.18 scanl1

```
scanl1 = lambda f, xs: scanl(f, xs)
```

2.1.19 scanr

```
scanr = lambda f, xs, base: scanl(flip(f), reversed(list(xs)), base)[::-1]
```

2.1.20 scanr1

```
scanr1 = lambda f, xs: scanl(flip(f), reversed(list(xs)))[::-1]
```

2.1.21 unfoldr

Anamorphism: $f(\text{seed}) \rightarrow (\text{value}, \text{seed}')$ or NOTHING to stop.

```
>>> unfoldr(lambda n: NOTHING if n == 0 else (n, n - 1), 4)
[4, 3, 2, 1]
```

```
def unfoldr(f, seed):
    out = []
    while (r := f(seed)) is not NOTHING:
        out.append(r[0]); seed = r[1]
    return out
```

2.1.22 until

Loop f until cond holds.

```
>>> until(lambda n: n > 100, lambda n: n * 2, 1)
128
```

```
def until(cond, f, x):
    while not cond(x):
        x = f(x)
    return x
```

2.1.23 replicateM

```
replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]])
```

2.1.24 subsequences

```
subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]])
```

2.1.25 iterate

Infinite stream $x, f(x), f(f(x)), \dots$

```
>>> take(5, iterate(lambda n: n * 2, 1))
[1, 2, 4, 8, 16]
```

```
def iterate(f, x):
    while True:
        yield x
        x = f(x)
```

2.1.26 take

```
take = lambda n, xs: list(_islice(iter(xs), n))
```

2.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

2.1 Parity census

drill

Count evens and odds as a pair (evens, odds), in ONE pass, point-free at the top level.

`parity_counts(xs) -> (evens, odds)`

```
>>> parity_counts([1, 2, 3, 4])
(2, 2)
>>> parity_counts([])
(0, 0)
>>> parity_counts([-2, -3])
(1, 1)
```

Solution.

```
parity_counts = partial(foldMap, lambda x: (1 - x % 2, x % 2), both(Sum, Sum))
```

Note. `both(Sum, Sum)` is two counters riding one fold; `partial` freezes the fold into a name. The `[]` case is free: monoid identity.

2.2 Running balance

drill

An account starts at 0; deposits are positive, withdrawals negative. Return every intermediate balance including the start.

`balances(txs) -> [balance]`

```
>>> balances([100, -30, -30, 5])
[0, 100, 70, 40, 45]
>>> balances([])
[0]
```

Solution.

```
balances = lambda txs: scanl(add, txs, 0)
```

Note. `Scan` = fold that keeps its diary. `operator.add` instead of a lambda.

2.3 Countdown

drill

Write `countdown(n)`: the list `n, n-1, ..., 1` grown by `unfoldr`, point-free at the top level.

`countdown(n) -> [int]`

```
>>> countdown(4)
[4, 3, 2, 1]
>>> countdown(0)
[]
```

Solution.

```
countdown = partial(unfoldr, lambda n: NOTHING if n == 0 else (n, n - 1))
```

Note. unfoldr is the anamorphism: seed in, list out, NOTHING stops it.

2.4 Digits of a number

applied

Write `digits(n)`: the base-10 digits of a non-negative integer, most significant first. Zero has one digit.

`digits(n) -> [int]`

```
>>> digits(407)
[4, 0, 7]
>>> digits(0)
[0]
```

Solution.

```
digits = lambda n: unfoldr(lambda k: NOTHING if k == 0 else (k % 10, k // 10), n)[::-1] or [0]
```

Note. Unfold peels least-significant first; reverse restores reading order. 'or [0]' patches the empty unfold at zero.

2.5 Collatz flight length

challenge

Write `collatz_len(n)`: how many numbers the Collatz sequence visits from `n` down to 1, inclusive. Even halves; odd triples-plus-one.

`collatz_len(n) -> int`

```
>>> collatz_len(6)
9
>>> collatz_len(1)
1
```

Solution.

```
collatz_len = lambda n: 1 + len(unfoldr(lambda k: NOTHING if k == 1 else (k, k // 2 if k % 2 == 0 else 3 * k +
    ↪ 1), n))
```

Note. The unfold emits everything before the fixed point; +1 counts the 1 itself. No mutation, no while.

Chapter 3

The List Gaps

3.1 Concepts

3.1.1 head

```
| head      = lambda xs: xs[0]
```

3.1.2 tail

```
| tail      = lambda xs: xs[1:]
```

3.1.3 init

```
| init      = lambda xs: xs[:-1]
```

3.1.4 last

```
| last      = lambda xs: xs[-1]
```

3.1.5 drop

```
| drop      = lambda n, xs: list(xs)[max(n, 0):]
```

3.1.6 splitAt

```
| splitAt   = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs)
```

3.1.7 replicate

```
| replicate = lambda n, x: [x] * n
```

3.1.8 elem

```
| elem      = lambda x, xs: x in xs
```

3.1.9 notElem

```
| notElem      = lambda x, xs: x not in xs
```

3.1.10 null

```
| null        = lambda xs: len(xs) == 0
```

3.1.11 nub

```
| nub         = o(list, dict.fromkeys)
```

3.1.12 enum

```
| enum        = lambda xs, start=0: list(enumerate(xs, start))
```

3.1.13 pairwise

```
| pairwise    = lambda xs: list(_pairwise(xs))
```

3.1.14 zip_

```
| zip_        = lambda a, b: list(zip(a, b))
```

3.1.15 zip3

```
| zip3        = lambda a, b, c: list(zip(a, b, c))
```

3.1.16 unzip

```
| unzip       = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], [])
```

3.1.17 isPrefixOf

```
| isPrefixOf  = lambda p, xs: list(xs[:len(p)]) == list(p)
```

3.1.18 isSuffixOf

```
| isSuffixOf  = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p)
```

3.1.19 lookup

Assoc-list lookup into Maybe (a stored None is data).

```
>>> lookup('b', [('a', 1), ('b', None)]) is None
True
>>> lookup('c', [('a', 1)]) is NOTHING
True
```

```
def lookup(k, pairs):
    return next((v for kk, v in pairs if kk == k), NOTHING)
```

3.1.20 stripPrefix

The rest after prefix p, or NOTHING.

```
>>> stripPrefix('foo', 'foobar')
'bar'
>>> stripPrefix('x', 'foobar') is NOTHING
True

def stripPrefix(p, xs):
    return xs[len(p):] if isPrefixOf(p, xs) else NOTHING
```

3.1.21 map_

```
| map_      = lambda f, xs: [f(x) for x in xs]
```

3.1.22 filter_

```
| filter_   = lambda cond, xs: [x for x in xs if cond(x)]
```

3.1.23 concat

```
| concat    = lambda xss: [x for xs in xss for x in xs]
```

3.1.24 concatMap

```
| concatMap = lambda f, xs: [y for x in xs for y in f(x)]
```

3.1.25 cross

```
| cross     = lambda a, b: [(x, y) for x in a for y in b]
```

3.1.26 starmap

```
| starmap   = lambda f, pairs: [f(*p) for p in pairs]
```

3.1.27 zipWith

```
| zipWith   = lambda f, a, b: [f(x, y) for x, y in zip(a, b)]
```

3.1.28 zipWith3

```
| zipWith3  = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)]
```

3.1.29 find

First match, lazily, into Maybe.

```
>>> find(even, [1, 3, 4, 5])
4
>>> find(even, [1, 3]) is NOTHING
True
```

```
def find(cond, xs):
    return next((x for x in xs if cond(x)), NOTHING)
```

3.1.30 partition

(keepers, rest) in ONE pass, cond called once per element.

```
>>> partition(even, [1, 2, 3, 4])
([2, 4], [1, 3])

def partition(cond, xs):
    yes, no = [], []
    for x in xs:
        (yes if cond(x) else no).append(x)
    return (yes, no)
```

3.1.31 span

Split at first failure, one pass, one cond call per element.

```
>>> span(lambda x: x < 3, [1, 2, 3, 1])
([1, 2], [3, 1])

def span(cond, xs):
    xs = list(xs)
    i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
    return (xs[:i], xs[i:])
```

3.1.32 break_

```
break_ = lambda cond, xs: span(o(not_), cond), xs)
```

3.1.33 takewhile

```
def takewhile(cond, xs):
    for x in xs:
        if not cond(x):
            return
    yield x
# generator
```

3.1.34 dropwhile

```
def dropwhile(cond, xs):
    it = iter(xs)
    for x in it:
        if not cond(x):
            yield x
            break
    yield from it
# generator
```

3.1.35 takeuntil

```
def takeuntil(cond, xs):
    for x in xs:
        yield x
        if cond(x):
            return
# takewhile that INCLUDES the first hit
```

3.1.36 takeWhile

```
| takeWhile = lambda cond, xs: list(takewhile(cond, xs))
```

3.1.37 dropWhile

```
| dropWhile = lambda cond, xs: list(dropwhile(cond, xs))
```

3.1.38 inits

```
| inits      = lambda xs: [xs[:i] for i in range(len(xs) + 1)]
```

3.1.39 tails

```
| tails      = lambda xs: [xs[i:] for i in range(len(xs) + 1)]
```

3.1.40 chunksOf

```
>>> chunksOf(2, [1, 2, 3, 4, 5])
[[1, 2], [3, 4], [5]]
```

```
def chunksOf(n, xs):
    xs = list(xs)
    return [xs[i:i + n] for i in range(0, len(xs), n)]
```

3.1.41 windows

Sliding windows of width n.

```
>>> windows(3, [1, 2, 3, 4])
[[1, 2, 3], [2, 3, 4]]
```

```
def windows(n, xs):
    xs = list(xs)
    return [xs[i:i + n] for i in range(len(xs) - n + 1)]
```

3.1.42 groupBy

Haskell semantics: runs anchored on each run's FIRST element.

```
>>> groupBy(lambda a, b: a == b, "aabba")
[['a', 'a'], ['b', 'b'], ['a']]
```

```
def groupBy(eq, xs):
    out = []
    for x in xs:
        if out and eq(out[-1][0], x):
            out[-1].append(x)
        else:
            out.append([x])
    return out
```

3.1.43 group

```
| group = partial(groupBy, lambda a, b: a == b)
```

3.1.44 transpose

Ragged-safe like Data.List.

```
>>> transpose([[1, 2, 3], [4, 5]])
[[1, 4], [2, 5], [3]]

def transpose(rows):
    rows = [list(r) for r in rows]
    out, i = [], 0
    while any(i < len(r) for r in rows):
        out.append([r[i] for r in rows if i < len(r)])
        i += 1
    return out
```

3.1.45 sortOn

```
sortOn = lambda f, xs: sorted(xs, key=f)
```

3.1.46 sortBy

```
sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp))
```

3.1.47 maxOn

```
maxOn = lambda f, xs: max(xs, key=f)
```

3.1.48 minOn

```
minOn = lambda f, xs: min(xs, key=f)
```

3.1.49 merge

```
merge = lambda a, b: list(_hmerge(a, b))
```

3.1.50 lines

```
lines = str.splitlines
```

3.1.51 unlines

```
unlines = lambda ls: "".join(l + "\n" for l in ls)
```

3.1.52 words

```
words = str.split
```

3.1.53 unwords

```
unwords = " ".join
```

3.1.54 fromListWith

Data.Map: THE dict-building fold; f(new, old).

```
>>> fromListWith(add, [('a', 1), ('b', 2), ('a', 3)]) == {'a': 4, 'b': 2}
True

def fromListWith(f, pairs):
    d = {}
    for k, v in pairs:
        d[k] = f(v, d[k]) if k in d else v
    return d
```

3.1.55 insertWith

Persistent insert (fresh dict); f(new, old).

```
>>> insertWith(add, 'a', 5, {'a': 1}) == {'a': 6}
True

def insertWith(f, k, v, d):
    return {**d, k: f(v, d[k]) if k in d else v}
```

3.1.56 unionWith

Data.Map: merge two dicts, combining shared keys with f.

```
>>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4}) == {'a': 1, 'b': 5, 'c': 4}
True

def unionWith(f, a, b):
    return {k: f(a[k], b[k]) if k in a and k in b else (a[k] if k in a else b[k])
            for k in a.keys() | b.keys()}
```

3.1.57 getpath

Read along a key path without autovivifying; Maybe out.

```
>>> getpath({'a': {'b': None}}, ['a', 'b']) is None
True
>>> getpath({'a': {}}, ['a', 'b']) is NOTHING
True

def getpath(t, ks, default=NOTHING):
    for k in ks:
        if not isinstance(t, dict) or k not in t:
            return default
        t = t[k]
    return t
```

3.1.58 bag_diff

```
bag_diff = lambda a, b: {k: a[k] - b.get(k, 0) for k in a if a[k] > b.get(k, 0)}
```

3.1.59 bag_inter

```
bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a[k], b[k]))}
```

3.1.60 bag_union

```
| bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
```

3.1.61 bag_sub

```
| bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items())
```

3.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

3.1 Run-length encoding

drill

Compress a sequence into (element, count) pairs per maximal run.

`rle(xs) -> [(x, n)]`

```
|>>> rle('aaabbc')
| [('a', 3), ('b', 2), ('c', 1)]
|>>> rle([])
| []
```

Solution.

```
| rle = lambda xs: [(g[0], len(g)) for g in groupBy(eq, xs)]
```

Note. `groupBy(eq, .)` does the segmentation; the comprehension just labels each run. `operator.eq`, no `lambda`.

3.2 Moving average

applied

Write `mavg(n, xs)`: the n -point moving average of a signal.

`mavg(n, xs) -> [float]`

```
|>>> mavg(3, [1, 2, 3, 4, 5])
| [2.0, 3.0, 4.0]
|>>> mavg(3, [1, 2])
| []
```

Solution.

```
| mavg = lambda n, xs: [sum(w) / n for w in windows(n, xs)]
```

Note. `windows` names the sliding view; the arithmetic stays arithmetic. Too-short input yields no windows – correct for free.

3.3 Pagination

drill

Write `paginate(n, xs)`: pages of at most n items, last page short.

`paginate(n, xs) -> [[x]]`

```
|>>> paginate(2, [1, 2, 3, 4, 5])
| [[1, 2], [3, 4], [5]]
```

Solution.

```
| paginate = chunksOf
```

Note. Recognize-it drill: pagination IS `chunksOf`.

3.4 Column sums, ragged

applied

Sensor rows arrive with trailing dropouts. Sum each column over the rows that reached it – point-free.

```
colsums(rows) -> [sum]
|>>> colsums([[1, 2, 3], [4, 5]])
|    [5, 7, 3]
|>>> colsums([])
|    []
```

Solution.

```
| colsums = o(list, partial(map, sum), transpose)
```

Note. Ragged-safe transpose turns dropouts into short columns instead of exceptions; then it is just map sum.

3.5 Indentation split

drill

Split a line into (leading whitespace, rest) as strings, one pass.

```
indent_split(s) -> (str, str)
|>>> indent_split('  x = 1')
|    ('  ', 'x = 1')
|>>> indent_split('top')
|    ('', 'top')
```

Solution.

```
| indent_split = lambda s: tuple(''.join(p) for p in span(str.isspace, s))
```

Note. span is the two-sided takewhile; the join restores strings from character lists.

Chapter 4

Maybe, Done Right

4.1 Concepts

4.1.1 bind

»= : short-circuit on NOTHING.

```
>>> bind(3, lambda v: v + 1), bind(NOTHING, lambda v: v + 1)
(4, NOTHING)
```

```
def bind(x, f):
    return NOTHING if x is NOTHING else f(x)
```

4.1.2 isJust

```
isJust = lambda x: x is not NOTHING
```

4.1.3 isNothing

```
isNothing = lambda x: x is NOTHING
```

4.1.4 fromMaybe

```
fromMaybe = lambda default, x: default if x is NOTHING else x
```

4.1.5 listToMaybe

```
listToMaybe = lambda xs: xs[0] if xs else NOTHING
```

4.1.6 maybeToList

```
maybeToList = lambda x: [] if x is NOTHING else [x]
```

4.1.7 catMaybes

```
>>> catMaybes([1, NOTHING, None, 2])
[1, None, 2]
```

```
def catMaybes(xs):
    return [x for x in xs if x is not NOTHING]
```

4.1.8 mapMaybe

Map into Maybe and drop the NOTHINGs, one pass.

```
>>> mapMaybe(lambda s: int(s) if s.isdigit() else NOTHING, ['3', 'x', '7'])
[3, 7]

def mapMaybe(f, xs):
    return [y for y in map(f, xs) if y is not NOTHING]
```

4.1.9 sequenceM

All-or-nothing.

```
>>> sequenceM([1, 2]), sequenceM([1, NOTHING])
([1, 2], NOTHING)

def sequenceM(xs):
    out = []
    for x in xs:
        if x is NOTHING:
            return NOTHING
        out.append(x)
    return out
```

4.1.10 traverseM

```
traverseM = lambda f, xs: sequenceM(map_(f, xs))
```

4.1.11 maybe_get

dict lookup into Maybe (None stays distinguishable from missing).

```
>>> maybe_get({'a': None}, 'a'), maybe_get({}, 'a')
(None, NOTHING)

def maybe_get(d, k):
    return d.get(k, NOTHING)
```

4.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

4.1 Safe head

drill

Write `headM(xs)`: the first element, or NOTHING for an empty sequence. A leading None is data.

`headM(xs) -> Maybe value`

```
>>> headM([7, 8])
7
>>> headM([]) is NOTHING
True
>>> headM([None]) is None
True
```

Solution.

```
headM = lambda xs: xs[0] if xs else NOTHING
```

Note. The third test is the whole point of the sentinel: `None` rides through as a value.

4.2 Deep dictionary path

applied

Write `pathM(d, ks)`: follow a key path through nested dicts; `NOTHING` if any hop is missing. A stored `None` must survive the trip.

`pathM(d, ks) -> Maybe value`

```
>>> d = {'db': {'host': 'local', 'pool': None}}
>>> pathM(d, ['db', 'host'])
'local'
>>> pathM(d, ['db', 'pool']) is None
True
>>> pathM(d, ['db', 'user']) is NOTHING
True
```

Solution.

```
| pathM = lambda d, ks: foldl(lambda m, k: bind(m, lambda t: maybe_get(t, k)), ks, d)
```

Note. A path lookup is a FOLD of binds over the keys – the Maybe monad’s signature move.

4.3 Salvage the parseable

drill

Given strings, keep the ones that are integers, as ints, dropping the rest.

`salvage_ints(ss) -> [int]`

```
>>> salvage_ints(['3', 'x', '-7', ''])
[3, -7]
```

Solution.

```
| salvage_ints = lambda ss: catMaybes([int(s) if s.lstrip('-').isdigit() and s else NOTHING for s in ss])
```

Note. Map into Maybe, then `catMaybes` sweeps the failures out – `mapMaybe` in two named steps.

4.4 All keys or nothing

applied

Write `need(d, ks)`: the values for `ks` in order, or `NOTHING` if ANY key is absent.

`need(d, ks) -> Maybe [value]`

```
>>> need({'a': 1, 'b': 2}, ['a', 'b'])
[1, 2]
>>> need({'a': 1}, ['a', 'b']) is NOTHING
True
```

Solution.

```
| need = lambda d, ks: sequenceM([maybe_get(d, k) for k in ks])
```

Note. `sequenceM` is the all-or-nothing gate: one miss poisons the batch.

4.5 First responder

challenge

Several config sources; write `first_set(dict1, dict2, ..., k)`: the value from the FIRST dict that defines `k` at all – even as `None` – else `NOTHING`.

`first_set(dict1, dict2, ..., k) -> Maybe value`

```
>>> first_set([{}], {'x': None}, {'x': 3}, 'x') is None
True
>>> first_set([{}], {}, 'x') is NOTHING
True
```

Solution.

```
| first_set = lambda dicts, k: mconcat(First, [maybe_get(d, k) for d in dicts])
```

Note. First's identity is NOTHING, so 'defined as None' beats 'undefined' – exactly the override semantics config layering needs.

Chapter 5

Either: Failure With a Why

5.1 Concepts

5.1.1 Ok

```
| Ok = lambda v: ("ok", v)
```

5.1.2 Err

```
| Err = lambda e: ("err", e)
```

5.1.3 bindE

```
| >>> bindE(Ok(3), lambda v: Ok(v + 1)), bindE(Err("no"), lambda v: Ok(v))  
| (('ok', 4), ('err', 'no'))  
  
| def bindE(x, f):  
|     return x if x[0] == "err" else f(x[1])
```

5.1.4 sequenceE

First Err wins; else Ok of all values.

```
| >>> sequenceE([Ok(1), Err("a"), Err("b")])  
| ('err', 'a')  
  
| def sequenceE(xs):  
|     out = []  
|     for x in xs:  
|         if x[0] == "err":  
|             return x  
|         out.append(x[1])  
|     return Ok(out)
```

5.1.5 traverseE

Map into Either, then sequence.

```
| >>> traverseE(lambda n: Ok(n * 2) if n else Err('zero'), [1, 2])  
| ('ok', [2, 4])  
  
| def traverseE(f, xs):  
|     return sequenceE(map_(f, xs))
```

5.1.6 partitionEithers

```
>>> partitionEithers([Ok(1), Err('a'), Ok(2)])
([1, 2], ['a'])

def partitionEithers(xs):
    oks, errs = [], []
    for tag, v in xs:
        (oks if tag == "ok" else errs).append(v)
    return (oks, errs)
```

5.1.7 note

Maybe -> Either: attach WHY to a NOTHING.

```
>>> note("missing key", NOTHING)
('err', 'missing key')

def note(msg, x):
    return Err(msg) if x is NOTHING else Ok(x)
```

5.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

5.1 Lift the miss into a reason

drill

Write `get_e(d, k): Ok(value) or Err naming the missing key`.

`get_e(d, k) -> Either`

```
>>> get_e({'a': 1}, 'a')
('ok', 1)
>>> get_e({}, 'user')
('err', "missing key: 'user'")
```

Solution.

```
get_e = lambda d, k: note(f"missing key: {k!r}", maybe_get(d, k))
```

Note. `note` is the Maybe-to-Either bridge: same short-circuit, now with a why.

5.2 Parse then bound

applied

Write `to_port(s): parse a decimal string and demand 1..65535, each failure carrying its own message`.

`to_port(s) -> Either`

```
>>> to_port('8080')
('ok', 8080)
>>> to_port('http')
('err', "not a number: 'http'")
>>> to_port('99999')
('err', 'out of range: 99999')
```

Solution.

```
to_port = lambda s: bindE(
    note(f"not a number: {s!r}", int(s) if s.isdigit() else NOTHING),
    lambda n: Ok(n) if 1 <= n <= 65535 else Err(f"out of range: {n}"))
```

Note. Two failure modes, two messages, one `bindE`. The caller pattern-matches once.

5.3 First error speaks

drill

Validate a batch: all Ok values, or the FIRST Err verbatim.

`all_ports(ss) -> Either`

```
>>> all_ports(['80', '443'])
('ok', [80, 443])
>>> all_ports(['80', 'x', 'y'])
('err', "not a number: 'x'")
```

Solution.

```
| all_ports = lambda ss: sequenceE([to_port(s) for s in ss])
```

Note. `sequenceE` = `sequenceM` that can explain itself.

5.4 Division with a why

drill

Write `div_e(a, b)`: the quotient, or an error naming the offense.

`div_e(a, b) -> Either`

```
>>> div_e(10, 4)
('ok', 2.5)
>>> div_e(1, 0)
('err', 'division by zero')
```

Solution.

```
| div_e = lambda a, b: Err('division by zero') if b == 0 else Ok(a / b)
```

Note. Exceptions escape; Either values travel. Choose Either at module boundaries.

Chapter 6

do-Notation

6.1 Concepts

6.1.1 do

Build a do-block decorator for any short-circuiting monad.

```
def do(binder, is_fail):
    def deco(gen_fn):
        def run(*a, **kw):
            g = gen_fn(*a, **kw)
            def step(val):
                try:
                    m = g.send(val)
                except StopIteration as e:
                    return e.value
            return binder(m, step)
        return step(None)
    return run
return deco
```

6.1.2 doM

```
doM = do(bind, lambda x: x is NOTHING)
```

6.1.3 doE

```
doE = do(bindE, lambda x: x[0] == "err")
```

6.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest` referee.

6.1 Config reader, flat

drill

Using doM, read host and port from a dict; NOTHING if either is absent.

hostport(d) -> Maybe (host, port)

```
>>> hostport({'host': 'h', 'port': 80})
('h', 80)
>>> hostport({'host': 'h'}) is NOTHING
```

```
| True
```

Solution.

```
@doM
def hostport(d):
    h = yield maybe_get(d, 'host')
    p = yield maybe_get(d, 'port')
    return (h, p)
```

Note. Compare the bind pyramid this replaces: each yield is one `»=`, and the failure path is invisible.

6.2 Join across two tables

applied

users maps id to name; emails maps name to address. Using doM, resolve an id to its address.

address(users, emails, uid) -> Maybe str

```
>>> u, e = {1: 'ada'}, {'ada': 'ada@x.io'}
>>> address(u, e, 1)
'ada@x.io'
>>> address(u, e, 2) is NOTHING
True
```

Solution.

```
@doM
def address(users, emails, uid):
    name = yield maybe_get(users, uid)
    return (yield maybe_get(emails, name))
```

Note. A two-hop join is two yields. The second bind consumes the first's value by name, not by nesting.

6.3 Validated record

applied

Using doE, build a user record from a form dict: name must be non-empty, age must parse and land in 0..130. Each rejection explains itself.

mk_user(form) -> Either dict

```
>>> mk_user({'name': 'ada', 'age': '36'})
('ok', {'name': 'ada', 'age': 36})
>>> mk_user({'name': '', 'age': '36'})
('err', 'empty name')
>>> mk_user({'name': 'ada', 'age': 'old'})
('err', "bad age: 'old'")
```

Solution.

```
@doE
def mk_user(form):
    name = yield (Ok(form.get('name', '')) if form.get('name') else Err('empty name'))
    raw = form.get('age', '')
    age = yield note(f"bad age: {raw!r}", int(raw) if raw.isdigit() else NOTHING)
    yield (Ok(age) if age <= 130 else Err(f'implausible age: {age}'))
    return Ok({'name': name, 'age': age})
```

Note. A bare yield with no binding is a guard: run the check, keep no value. The happy path reads top to bottom.

6.4 Chained arithmetic

drill

Using doE and div_e from 5.4, compute $a/b + c/d$ with the first bad divisor reported.

sum_of_ratios(a, b, c, d) -> Either

```
>>> sum_of_ratios(1, 2, 1, 4)
('ok', 0.75)
```

```
>>> sum_of_ratios(1, 0, 1, 4)
('err', 'division by zero')
```

Solution.

```
@doE
def sum_of_ratios(a, b, c, d):
    x = yield div_e(a, b)
    y = yield div_e(c, d)
    return Ok(x + y)
```

Note. Error plumbing gone; only the arithmetic remains.

Chapter 7

Parser Combinators

7.1 Concepts

7.1.1 doP

do-notation that THREADS the input: yield a parser, receive its value. Fresh generator per parse – re-runnable, backtrack-safe under alt.

```
def doP(gen_fn):
    def make(*a, **kw):
        def parser(s):
            g, val = gen_fn(*a, **kw), None
            try:
                while True:
                    r = g.send(val)(s)
                    if r is FAIL:
                        return FAIL
                    val, s = r
            except StopIteration as e:
                return (e.value, s)
        return parser
    return make
```

7.1.2 alt

<|> : first success wins.

```
def alt(*ps):
    return lambda s: next((r for r in (p(s) for p in ps) if r is not FAIL), FAIL)
```

7.1.3 many

Zero or more, loop not recursion; stalls guard against empty matches.

```
def many(p):
    def parser(s):
        out = []
        while (r := p(s)) is not FAIL and r[1] != s:
            out.append(r[0]); s = r[1]
        return (out, s)
    return parser
```

7.1.4 many1

```
def many1(p):
    def parser(s):
        r = p(s)
        if r is FAIL:
            return FAIL
        rest = many(p)(r[1])
        return ([r[0]] + rest[0], rest[1])
    return parser
```

7.1.5 sepBy

```
def sepBy(p, sep):
    def pair(s):
        r = sep(s)
        return FAIL if r is FAIL else p(r[1])
    def parser(s):
        r = p(s)
        if r is FAIL:
            return ([], s)
        rest = many(pair)(r[1])
        return ([r[0]] + rest[0], rest[1])
    return parser
```

7.1.6 lit

One expected character, whitespace-blind.

```
def lit(c):
    def parser(s):
        s = s.lstrip()
        return (c, s[1:]) if s[:1] == c else FAIL
    return parser
```

7.1.7 rx

Regex token, whitespace-blind, converted.

```
def rx(pat, conv=identity):
    r = _re.compile(pat)
    def parser(s):
        s = s.lstrip()
        m = r.match(s)
        return (conv(m.group()), s[m.end():]) if m else FAIL
    return parser
```

7.1.8 chain1

$p \text{ (op } p)^*$ folded LEFT; ops maps operator char -> binary function.

```
def chain1(p, ops):
    opp = alt(*[lit(c) for c in ops])
    def parser(s):
        vr = p(s)
        while vr is not FAIL:
            op = opp(vr[1])
            if op is FAIL:
                return vr
            w = p(op[1])
            if w is FAIL:
                return vr
```

```

        vr = (ops[op[0]](vr[0], w[0]), w[1])
    return FAIL
return parser

```

7.1.9 runParser

Parser -> Either: Ok(value) or Err showing where it stopped.

```

>>> runParser(rx(r'\d+', int), "42")
('ok', 42)
>>> runParser(rx(r'\d+', int), "42x")
('err', "unconsumed input: 'x'")

def runParser(p, s):
    r = p(s)
    if r is FAIL:
        return Err(f"no parse: {s!r}")
    if r[1].strip():
        return Err(f"unconsumed input: {r[1].strip()!r}")
    return Ok(r[0])

```

7.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

7.1 Comma-separated integers

drill

Parse a comma-separated integer list, whitespace-blind, whole input consumed.

`int_list(s) -> Either [int]`

```

>>> int_list('1, -2 , 3')
('ok', [1, -2, 3])
>>> int_list('1, x')
('err', "unconsumed input: ', x'")

```

Solution.

```
int_list = lambda s: runParser(sepBy(rx(r'?\d+', int), lit(',')), s)
```

Note. `sepBy` handles the fencepost (separators BETWEEN items); `runParser` demands the input be spent and lands in `Either`.

7.2 key=value line

drill

Parse one 'key = value' assignment into a pair; keys are words, values run to end of line.

`kv(s) -> Either (k, v)`

```

>>> kv('host = 10.0.0.1')
('ok', ('host', '10.0.0.1'))
>>> kv('= oops')[0]
'err'

```

Solution.

```

@doP
def _kv():
    k = yield rx(r'\w+')
    yield lit('=')
    v = yield rx(r'^\n+', str.strip)
    return (k, v)
kvline = _kv()

```

```
| kv = lambda s: runParser(kvline, s)
```

Note. Three yields, no rest-threading anywhere. kvline is reusable – chapter 9’s INI reader is built on it.

7.3 CSV row

drill

Split one unquoted CSV row into stripped fields.

```
csv_row(s) -> Either [str]

|>>> csv_row('ada, 95, pass')
|('ok', ['ada', '95', 'pass'])
```

Solution.

```
| csv_row = lambda s: runParser(sepBy(rx(r'[^,\n]+'), str.strip), lit(','), s)
```

Note. The same sepBy skeleton as 7.1 with a different token – combinator reuse is the whole pitch.

7.4 ISO date

applied

Parse YYYY-MM-DD into an (y, m, d) integer triple, rejecting month 13.

```
iso_date(s) -> Either (y, m, d)

|>>> iso_date('2026-09-17')
|('ok', (2026, 9, 17))
|>>> iso_date('2026-13-01')
|('err', 'bad month: 13')
```

Solution.

```
num = lambda n: rx(r'\d{%d}' % n, int)
@doP
def _date():
    y = yield num(4)
    yield lit('-')
    m = yield num(2)
    yield lit('-')
    d = yield num(2)
    return (y, m, d)
iso_date = lambda s: bindE(runParser(_date(), s),
    lambda t: Ok(t) if 1 <= t[1] <= 12 else Err(f'bad month: {t[1]}'))
```

Note. Parse the shape with doP, judge the meaning with bindE – syntax and semantics in separate layers.

7.5 Sums and differences

challenge

Parse and evaluate a left-associative +/- chain over integers.

```
sums(s) -> Either int

|>>> sums('10 - 3 - 2 + 1')
|('ok', 6)
|>>> sums('10 -')[0]
|'err'
```

Solution.

```
| sums = lambda s: runParser(chainl1(rx(r'[-?\d+]', int), {'+': add, '-': sub}), s)
```

Note. chainl1 IS left-associativity-as-a-fold; operator.add/sub keep it point-free. Chapter 9 grows this into the full calculator.

Chapter 8

Monoids at Work

8.1 Concepts

8.1.1 Monoid

```
| Monoid = lambda empty, op: (empty, op)
```

8.1.2 mconcat

```
| mconcat = lambda m, xs: foldl(m[1], xs, m[0])
```

8.1.3 foldMap

```
| foldMap = lambda f, m, xs: mconcat(m, [f(x) for x in xs])
```

8.1.4 both

```
| both = lambda m1, m2: Monoid((m1[0], m2[0]),  
                                lambda a, b: (m1[1](a[0], b[0]), m2[1](a[1], b[1])))
```

8.1.5 Sum

```
| Sum = Monoid(0, add)
```

8.1.6 Product

```
| Product = Monoid(1, mul)
```

8.1.7 All

```
| All = Monoid(True, lambda a, b: a and b)
```

8.1.8 Any

```
| Any = Monoid(False, lambda a, b: a or b)
```

8.1.9 MaxM

```
| MaxM    = Monoid(float("-inf"), max)
```

8.1.10 MinM

```
| MinM    = Monoid(float("inf"), min)
```

8.1.11 ListM

```
| ListM   = Monoid([], add)
```

8.1.12 First

```
| First   = Monoid(NOTHING, lambda a, b: b if a is NOTHING else a)
```

8.1.13 Last

```
| Last    = Monoid(NOTHING, lambda a, b: a if b is NOTHING else b)
```

8.2 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

8.1 Min and max, one pass

drill

Both extremes of a numeric stream in a single traversal, point-free.

```
extremes(xs) -> (min, max)
```

```
>>> extremes([3, 1, 4, 1, 5])
(1, 5)
```

Solution.

```
| extremes = partial(foldMap, lambda x: (x, x), both(MinM, MaxM))
```

Note. both fuses two folds into one pass – what matters when xs is a one-shot iterator or a large file.

8.2 Stream statistics

applied

Count, total and maximum of a stream in ONE pass: ((count, total), maximum).

```
stats(xs) -> ((n, total), mx)
```

```
>>> stats([3, 1, 4])
((3, 8), 4)
```

Solution.

```
| stats = partial(foldMap, lambda x: ((1, x), x), both(both(Sum, Sum), MaxM))
```

Note. both nests: a monoid of monoids. The tuple shape of the answer mirrors the shape of the fold.

8.3 Config layering

drill

defaults, file, cli – highest layer that DEFINES a key wins, and defining it as None counts. Point-free over the layer list.

```
effective(layers, k) -> Maybe value
|>>> layers = [{'v': 1}, {}, {'v': None}]
|>>> effective(layers, 'v') is None
|True
|>>> effective(layers, 'x') is NOTHING
|True
```

Solution.

```
| effective = lambda layers, k: mconcat>Last, [maybe_get(d, k) for d in layers])
```

Note. Last with a NOTHING identity is override semantics verbatim. Same skeleton as 4.5 with the opposite monoid.

8.4 Flatten

drill

Concatenate a list of lists by monoid fold.

```
flatten(xss) -> [x]
|>>> flatten([[1], [], [2, 3]])
|[1, 2, 3]
```

Solution.

```
| flatten = partial(mconcat, ListM)
```

Note. Recognize-it: concatenation is the list monoid's mconcat. (For huge inputs reach for `itertools.chain` – this one is quadratic.)

Chapter 9

Capstone Labs

9.1 Homework

Work in order: drills cement each tool, applies combine them, challenges are interview-grade. Write each solution in a scratch file with the given doctests pasted in, and let `python3 -m doctest referee`.

9.1 The algebraic calculator

lab

Full expression evaluator: `+ - * /`, precedence, LEFT associativity, unary minus, parentheses, decimals, whitespace; Either out.
`calc(s) -> Either float`

```
>>> calc('2 * (3 + 4) - -5')
('ok', 19.0)
>>> calc('8 - 3 - 2')
('ok', 3.0)
>>> calc('2 * (3 +)')[0]
'err'
```

Solution.

```
number = rx(r'[-?\d+(\.\d+)??', float)
@doP
def _paren():
    yield lit('(')
    v = yield expr
    yield lit(')')
    return v
unary = lambda s: alt(number, _paren())(s)
term = chainl1(unary, {'*': mul, '/': lambda a, b: a / b})
expr = chainl1(term, {'+': add, '-': sub})
calc = partial(runParser, expr)
```

Note. Eight grammar lines. Every layer earns its keep: `doP` threads state, `chainl1` folds associativity, `runParser` explains failure. 5000-term chains run in constant stack.

9.2 INI reader

lab

Parse `[section]` headers with `key = value` bodies into a dict of dicts.

`ini(s) -> Either section: k: v`

```
>>> t = '[db]\nhost = local\nport = 5432\n[app]\ndebug = true'
>>> ini(t)
('ok', {'db': {'host': 'local', 'port': '5432'}, 'app': {'debug': 'true'}})
```

Solution.

```
@doP
```

```

def _kvi():
    k = yield rx(r'\w+')
    yield lit('=')
    v = yield rx(r'[\n\[]+', str.strip)
    return (k, v)
@doP
def _section():
    yield lit('[')
    name = yield rx(r'\w+')
    yield lit(']')
    kvs = yield many(_kvi())
    return (name, dict(kvs))
ini = lambda s: bindE(runParser(many(_section()), s), o(Ok, dict))

```

Note. The 7.2 skeleton, nested once. many loops, so thousand-key files cost no stack; bindE lifts the final dict without unwrapping by hand.

9.3 Log triage

lab

Lines are 'LEVEL message'. In ONE pass report ((errors, total), first_ERROR message or NOTHING).

trriage(lines) -> ((errors, total), first_err)

```

>>> triage(['INFO up', 'ERROR db down', 'ERROR retry', 'INFO ok'])
((2, 4), 'db down')
>>> triage(['INFO up'])[1] is NOTHING
True

```

Solution.

```

tag = lambda ln: (lambda lvl, _, msg: ((lvl == 'ERROR', 1),
    msg if lvl == 'ERROR' else NOTHING))(*ln.partition(' '))
trriage = partial(foldMap, tag, both(both(Sum, Sum), First))

```

Note. Counting and first-witness are DIFFERENT monoids riding the same pass. Adding a new statistic is one more both, not another loop.

9.4 Batch admission

lab

Admit a batch of form dicts with 6.3's mk_user: all records, or the first rejection verbatim.

admit(forms) -> Either [dict]

```

>>> admit([{'name': 'ada', 'age': '36'}, {'name': 'bob', 'age': '41'}])
('ok', [{'name': 'ada', 'age': 36}, {'name': 'bob', 'age': 41}])
>>> admit([{'name': 'ada', 'age': '36'}, {'name': '', 'age': '2'}])
('err', 'empty name')

```

Solution.

```

admit = o(sequenceE, partial(map, mk_user), list)

```

Note. Validation composes: per-record doE, batch-level sequenceE, glued point-free. The whole intake pipeline is one line.

Part II

Compare and Contrast: Imperative vs Functional

About this part

Ten small apps, each written twice over the same inputs: once imperative, once with `haskell.py`, with a `__main__` block that ASSERTS the two agree. Every listing below is verified to actually run and agree before this book is built – what you read is what was executed.

	imperative	functional
state	mutable slots you maintain	threaded by the combinator
errors	exceptions / early return	values (NOTHING, Err) that compose
no value yet	hand-rolled sentinel per site	NOTHING, once, in the library
passes over data	one loop per concern	monoids fuse concerns into one pass
extension	edit inside the loop	add a stage / another <code>both</code>

Chapter 10

calculator

evaluate '2 * (3 + 4) - -5' both ways

Imperative

```
def calc_imp(s):
    toks, i = [], 0
    while i < len(s):
        c = s[i]
        if c.isspace(): i += 1; continue
        if c in '+-*/()': toks.append(c); i += 1; continue
        if c.isdigit():
            j = i
            while j < len(s) and (s[j].isdigit() or s[j] == '.'): j += 1
            toks.append(float(s[i:j])); i = j; continue
        return ('err', f'bad char {c!r}')

    class P:
        pos = 0
        def peek(): return toks[P.pos] if P.pos < len(toks) else None
        def unary():
            t = peek()
            if t == '-': P.pos += 1; return -unary()
            if t == '(':
                P.pos += 1; v = expr()
                if peek() != ')': raise ValueError('expected ')
                P.pos += 1; return v
            if isinstance(t, float): P.pos += 1; return t
            raise ValueError(f'unexpected {t!r}')
        def term():
            v = unary()
            while peek() in ('*', '/'):
                op = peek(); P.pos += 1; w = unary()
                v = v * w if op == '*' else v / w
            return v
        def expr():
            v = term()
            while peek() in ('+', '-'):
                op = peek(); P.pos += 1; w = term()
                v = v + w if op == '+' else v - w
            return v

    try:
        v = expr()
        return ('ok', v) if P.pos == len(toks) else ('err', 'unconsumed input')
    except ValueError as e:
```

```
return ('err', str(e))
```

Functional

```
number = rx(r'[-?\d+(\.\d+)?', float)
@doP
def _paren():
    yield lit('(')
    v = yield expr_fn
    yield lit(')')
    return v
unary_fn = lambda s: alt(number, _paren())(s)
term_fn = chainl1(unary_fn, {'*': mul, '/': lambda a, b: a / b})
expr_fn = chainl1(term_fn, {'+': add, '-': sub})
calc_fn = partial(runParser, expr_fn)
```

Chapter 11

gradebook

validate 'name,score' rows, then one-pass class stats

Imperative

```
def admit_imp(rows):
    out = []                                # accumulate + early return
    for row in rows:
        name, _, raw = row.partition(',')
        if not name:
            return ('err', f'empty name in {row!r}')
        if not raw.strip().isdigit():
            return ('err', f'bad score in {row!r}')
        score = int(raw)
        if score > 100:
            return ('err', f'score > 100 in {row!r}')
        out.append((name, score))
    n = tot = 0                             # second loop for stats
    best = float('-inf')
    for _, s in out:
        n += 1; tot += s
        if s > best: best = s
    return ('ok', (out, (n, tot), best))
```

Functional

```
@doE
def _row(row):
    name, _, raw = row.partition(',')
    yield (Ok(name) if name else Err(f'empty name in {row!r}'))
    s = yield note(f'bad score in {row!r}', int(raw) if raw.strip().isdigit() else NOTHING)
    yield (Ok(s) if s <= 100 else Err(f'score > 100 in {row!r}'))
    return Ok((name, s))

stats = partial(foldMap, lambda p: ((1, p[1]), p[1]), both(both(Sum, Sum), MaxM))
admit_fn = lambda rows: bindE(traverseE(_row, rows),
                              lambda ps: Ok((ps, *stats(ps))))
```


Chapter 12

layers

layered config: defaults < file < cli; None is a real value

Imperative

```
_UNSET = object()                                # invent a private sentinel,
def effective_imp(layers, k):                      # because None must survive
    val = _UNSET
    for d in layers:
        if k in d:
            val = d[k]
    return NOTHING if val is _UNSET else val

def merged_imp(layers):
    out = {}
    for d in layers:
        for k, v in d.items():
            out[k] = v
    return out
```

Functional

```
effective_fn = lambda layers, k: mconcat(Last, [maybe_get(d, k) for d in layers])
merged_fn    = lambda layers: foldl(lambda a, b: unionWith(lambda old, new: new, a, b),
                                   layers, {})
```


Chapter 13

leaderboard

clamp scores into [0, 100], rank by score desc, name asc

Imperative

```
def rank_imp(rows):
    clamped = []
    for name, score in rows:
        if score < 0:
            score = 0
        elif score > 100:
            score = 100
        clamped.append((name, score))
    clamped.sort(key=lambda r: (-r[1], r[0]))      # score desc, name asc
    return clamped
```

Functional

```
clamp = lambda lo, hi: o(partial(max, lo), partial(min, hi))
clamp100 = clamp(0, 100)
name_of, score_of = itemgetter(0), itemgetter(1)
rank_fn = lambda rows: sortOn(
    lambda r: (-score_of(r), name_of(r)),
    [(name_of(r), clamp100(score_of(r))) for r in rows])
```


Chapter 14

logtrriage

'LEVEL message' lines: error count, total, first error message

Imperative

```
def triage_imp(lines):
    errors = 0                                # three mutable slots
    total = 0
    first = None                              # and a hand-rolled sentinel:
    seen_first = False                       # None alone can't mean "none yet"
    for ln in lines:                          # if a message could be empty
        lvl, _, msg = ln.partition(' ')
        total += 1
        if lvl == 'ERROR':
            errors += 1
            if not seen_first:
                first, seen_first = msg, True
    return ((errors, total), first if seen_first else NOTHING)
```

Functional

```
tag = lambda ln: (lambda lvl, _, msg:
    ((lvl == 'ERROR', 1), msg if lvl == 'ERROR' else NOTHING)
   )(*ln.partition(' '))
triage_fn = partial(foldMap, tag, both(both(Sum, Sum), First))
```


Chapter 15

orderbatch

capstone: validate a batch of orders ($\text{qty} > 0$, $\text{price} > 0$), computing each line total; the whole batch fails at the first bad order, carrying which one and why

Imperative

```
def total_imp(order):
    if order["qty"] <= 0:
        return ("err", f"{order['item']}: bad qty {order['qty']}")
    if order["price"] <= 0:
        return ("err", f"{order['item']}: bad price {order['price']}")
    return ("ok", round(order["qty"] * order["price"], 2))

def admit_imp(orders):
    out = []
    for o in orders:
        r = total_imp(o)
        if r[0] == "err":
            return r
        out.append(r[1])
    return ("ok", out)
```

Functional

```
@doE
def total_fn(order):
    qty = yield (Ok(order["qty"]) if order["qty"] > 0
                else Err(f"{order['item']}: bad qty {order['qty']}"))
    price = yield (Ok(order["price"]) if order["price"] > 0
                  else Err(f"{order['item']}: bad price {order['price']}"))
    return Ok(round(qty * price, 2))

admit_fn = lambda orders: sequenceE([total_fn(o) for o in orders])
```


Chapter 16

orgchart

resolve an employee's manager's manager's email, or NOTHING if any hop is missing. None-as-Nothing means a missing hop and a real missing-email must stay distinguishable

Imperative

```
def skip_manager_email_imp(emp):
    mgr = MANAGER_OF.get(emp, NOTHING)
    if mgr is NOTHING:
        return NOTHING
    skip = MANAGER_OF.get(mgr, NOTHING)
    if skip is NOTHING:
        return NOTHING
    if skip not in EMAIL_OF:
        return NOTHING
    return EMAIL_OF[skip]
```

Functional

```
@doM
def skip_manager_email_fn(emp):
    mgr = yield maybe_get(MANAGER_OF, emp)
    skip = yield maybe_get(MANAGER_OF, mgr)
    return (yield maybe_get(EMAIL_OF, skip))
```


Chapter 17

payoff

loan payoff schedule: balances after each fixed payment, and how many payments until paid off

Imperative

```
def schedule_imp(balance, payment, rate):
    out, n = [balance], 0
    while balance > 0:
        balance = round(balance * (1 + rate) - payment, 2)
        if balance < 0:
            balance = 0.0
        out.append(balance)
        n += 1
    return out, n
```

Functional

```
def _step(payment, rate):
    def step(balance):
        if balance <= 0:
            return NOTHING
        nxt = round(balance * (1 + rate) - payment, 2)
        nxt = max(nxt, 0.0)
        return (nxt, nxt)
    return step

def schedule_fn(balance, payment, rate):
    rest = unfoldr(_step(payment, rate), balance)
    return [balance] + rest, len(rest)
```


Chapter 18

sensoralert

smooth a sensor stream with a 3-wide moving average, then report each consecutive run of over-threshold readings as (start, len)

Imperative

```
def alerts_imp(xs, n, threshold):
    smoothed = []
    for i in range(len(xs) - n + 1):
        window = xs[i:i + n]
        smoothed.append(sum(window) / n)
    flags = [v > threshold for v in smoothed]
    runs, i = [], 0
    while i < len(flags):
        if flags[i]:
            start = i
            while i < len(flags) and flags[i]:
                i += 1
            runs.append((start, i - start))
        else:
            i += 1
    return runs
```

Functional

```
def alerts_fn(xs, n, threshold):
    smoothed = [sum(w) / n for w in windows(n, xs)]
    flagged = [(i, v > threshold) for i, v in enum(smoothed)]
    groups = groupBy(lambda a, b: a[1] == b[1], flagged)
    return [(g[0][0], len(g)) for g in groups if g[0][1]]
```


Chapter 19

wordfreq

top-N word frequencies, ties alphabetical

Imperative

```
def top_imp(n, text):
    counts = {}                                # mutate a dict
    for w in text.lower().split():
        if w in counts:
            counts[w] += 1
        else:
            counts[w] = 1
    items = list(counts.items())                # then sort, then slice
    items.sort(key=lambda kv: (-kv[1], kv[0]))
    return items[:n]
```

Functional

```
top_fn = lambda n, text: pipe(
    text, str.lower, words,
    lambda ws: fromListWith(add, [(w, 1) for w in ws]),
    dict.items, partial(sortOn, lambda kv: (-kv[1], kv[0])),
    partial(take, n))
```


Part III

Appendices

Appendix A

haskell.py, Complete

```
1  """haskell.py -- the comprehensive Haskell-flavoured toolbox for Python.
2
3  One importable vocabulary: the working parts of the Prelude, Data.List,
4  Data.Maybe, Data.Either, Data.Map, Data.Function, Control.Monad, monoids,
5  and parser combinators, folded into sections and mastered as one language.
6
7  Design rules
8  * Where the stdlib already has it, the Haskell name WRAPS the stdlib
9    (scanl is itertools.accumulate, memo is functools.cache) -- one
10   vocabulary, zero reimplementation.
11  * Failure is a value. Maybe uses the true NOTHING sentinel, so None is
12    LEGAL data everywhere; parsers fail with FAIL, and Either carries why.
13  * do-notation via generators: `yield` is >=> (linear monads only).
14  * Constant stack wherever Haskell would lean on TCO: loops, not recursion.
15  * Python keywords and builtins keep prelude.py's underscore convention
16    (not_, map_, filter_, zip_); `id` stays Python's -- use identity.
17  """
18  from functools import reduce as _reduce, partial, cache as memo, cmp_to_key
19  from itertools import (accumulate as _accumulate, islice as _islice,
20                        chain, count, cycle, repeat, pairwise as _pairwise)
21  from operator import add, sub, mul, not_
22  from math import gcd, lcm, hypot, isqrt, prod as product
23  from collections import Counter, defaultdict, deque
24  from heapq import heappush, heappop, merge as _hmerge
25  import re as _re
26
27  # ===== 1. functions =====
28  identity = lambda x: x
29  const    = lambda x: lambda _: x
30  flip     = lambda f: lambda a, b: f(b, a)
31  curry    = lambda f: lambda x: lambda y: f(x, y)
32  uncurry  = lambda f: lambda p: f(*p)
33
34  def compose(*fs):
35      """Right-to-left composition, iterative (constant stack).
36      >>> compose(str, abs)(-3)
37      '3'
38      >>> o(str, abs)(-3)
39      '3'
40      """
41      def g(x):
42          for f in reversed(fs):
43              x = f(x)
44          return x
45      return g
46
47  o = compose                # ML's operator: o(f, g) is f after g
48
49  def pipe(x, *fs):
50      """Left-to-right application: pipe(x, f, g) == g(f(x)).
51      >>> pipe(-3, abs, str)
52      '3'
53      """
```

```

54     for f in fs:
55         x = f(x)
56     return x
57
58 def on(f, g):
59     """Data.Function: combine two arguments via a key.
60     >>> on(sub, len)('haskell', 'py')
61     5
62     """
63     return lambda x, y: f(g(x), g(y))
64
65 # ===== 2. pairs =====
66 fst = lambda p: p[0]
67 snd = lambda p: p[1]
68 swap = o(tuple, reversed)
69
70 # ===== 3. sentinels =====
71 class _Sentinel:
72     __slots__ = ("_n",)
73     def __init__(self, n): self._n = n
74     def __repr__(self): return self._n
75     def __bool__(self): return False
76
77 NOTHING = _Sentinel("NOTHING") # Maybe's Nothing; None is a legal Just here
78 FAIL = _Sentinel("FAIL") # parser failure; (None, rest) is a legal success
79 _MISS = _Sentinel("_MISS") # internal: absent optional argument
80
81 # ===== 4. arithmetic & logic =====
82 # add, sub, mul, not_ are operator's; gcd, lcm, hypot, isqrt, product are math's
83 div = lambda a, b: a // b # floors, like Haskell div
84 mod = lambda a, b: a % b # sign follows b
85 halve = lambda n: n // 2
86 even = lambda n: n % 2 == 0
87 odd = lambda n: n % 2 == 1
88 otherwise = True
89
90 def quot(a, b):
91     """Truncating division, Haskell quot; rem's sign follows a.
92     >>> quotRem(-7, 2)
93     (-3, -1)
94     """
95     return -(-a // b) if (a < 0) != (b < 0) else a // b
96
97 rem = lambda a, b: a - b * quot(a, b)
98 quotRem = lambda a, b: (quot(a, b), rem(a, b))
99 signum = lambda x: (x > 0) - (x < 0)
100 succ = lambda x: chr(ord(x) + 1) if isinstance(x, str) else x + 1
101 pred = lambda x: chr(ord(x) - 1) if isinstance(x, str) else x - 1
102
103 # ===== 5. folds, scans, unfolds =====
104 def foldl(f, xs, base=_MISS):
105     """foldl; seeds with the first element if no base is given.
106     >>> foldl(lambda a, b: a - b, [1, 2, 3], 10)
107     4
108     """
109     return _reduce(f, xs) if base is _MISS else _reduce(f, xs, base)
110
111 def foldr(f, xs, base):
112     """Right fold without recursion.
113     >>> foldr(lambda x, acc: [x] + acc, [1, 2, 3], [])
114     [1, 2, 3]
115     """
116     return _reduce(flip(f), reversed(list(xs)), base)
117
118 foldl1 = foldl
119 foldr1 = lambda f, xs: _reduce(flip(f), reversed(list(xs)))
120
121 def scanl(f, xs, base=_MISS):
122     """All intermediate foldl states (itertools.accumulate).
123     >>> scanl(add, [1, 2, 3], 0)
124     [0, 1, 3, 6]
125     """
126     return list(_accumulate(xs, f) if base is _MISS
127                 else _accumulate(xs, f, initial=base))

```

```

127
128 scanl1 = lambda f, xs: scanl(f, xs)
129 scanr  = lambda f, xs, base: scanl(flip(f), reversed(list(xs)), base[::-1])
130 scanr1 = lambda f, xs: scanl(flip(f), reversed(list(xs)))[::-1]
131
132 def unfoldr(f, seed):
133     """Anamorphism: f(seed) -> (value, seed') or NOTHING to stop.
134     >>> unfoldr(lambda n: NOTHING if n == 0 else (n, n - 1), 4)
135     [4, 3, 2, 1]
136     """
137     out = []
138     while (r := f(seed)) is not NOTHING:
139         out.append(r[0]); seed = r[1]
140     return out
141
142 def until(cond, f, x):
143     """Loop f until cond holds.
144     >>> until(lambda n: n > 100, lambda n: n * 2, 1)
145     128
146     """
147     while not cond(x):
148         x = f(x)
149     return x
150
151 # brute-force enumerations (say the bound out loud):
152 replicateM = lambda n, xs: foldl(lambda acc, _: [s + [x] for s in acc for x in xs], range(n), [[]]) # |xs|^n
153 subsequences = lambda xs: foldl(lambda acc, x: acc + [s + [x] for s in acc], list(xs), [[]]) # 2^n
154
155 # ===== 6. streams ===== (lazy; chain, count, cycle, repeat are itertools')
156 def iterate(f, x):
157     """Infinite stream x, f(x), f(f(x)), ...
158     >>> take(5, iterate(lambda n: n * 2, 1))
159     [1, 2, 4, 8, 16]
160     """
161     while True:
162         yield x
163         x = f(x)
164
165 take = lambda n, xs: list(_islice(iter(xs), n)) # safe on infinite streams
166
167 # ===== 7. list basics =====
168 head = lambda xs: xs[0]
169 tail = lambda xs: xs[1:]
170 init = lambda xs: xs[:-1]
171 last = lambda xs: xs[-1]
172 drop = lambda n, xs: list(xs)[max(n, 0):]
173 splitAt = lambda n, xs: (xs[:n], xs[n:]) if n >= 0 else (xs[:0], xs)
174 replicate = lambda n, x: [x] * n
175 elem = lambda x, xs: x in xs
176 notElem = lambda x, xs: x not in xs
177 null = lambda xs: len(xs) == 0
178 nub = o(list, dict.fromkeys) # unique, first seen wins
179 enum = lambda xs, start=0: list(enumerate(xs, start))
180 pairwise = lambda xs: list(_pairwise(xs))
181 zip_ = lambda a, b: list(zip(a, b))
182 zip3 = lambda a, b, c: list(zip(a, b, c))
183 unzip = lambda ps: tuple(map(list, zip(*ps))) if ps else ([], [])
184 isPrefixOf = lambda p, xs: list(xs[:len(p)]) == list(p)
185 isSuffixOf = lambda p, xs: list(xs[len(xs) - len(p):]) == list(p)
186
187 def lookup(k, pairs):
188     """Assoc-list lookup into Maybe (a stored None is data).
189     >>> lookup('b', [(a, 1), (b, None)]) is None
190     True
191     >>> lookup('c', [(a, 1)]) is NOTHING
192     True
193     """
194     return next((v for kk, v in pairs if kk == k), NOTHING)
195
196 def stripPrefix(p, xs):
197     """The rest after prefix p, or NOTHING.
198     >>> stripPrefix('foo', 'foobar')
199     'bar'

```

```

200 >>> stripPrefix('x', 'foobar') is NOTHING
201 True
202 """
203 return xs[len(p):] if isPrefixOf(p, xs) else NOTHING
204
205 # ===== 8. higher-order lists =====
206 map_      = lambda f, xs: [f(x) for x in xs]
207 filter_   = lambda cond, xs: [x for x in xs if cond(x)]
208 concat    = lambda xss: [x for xs in xss for x in xs]
209 concatMap = lambda f, xs: [y for x in xs for y in f(x)]
210 cross     = lambda a, b: [(x, y) for x in a for y in b]
211 starMap   = lambda f, pairs: [f(*p) for p in pairs]
212 zipWith   = lambda f, a, b: [f(x, y) for x, y in zip(a, b)]
213 zipWith3  = lambda f, a, b, c: [f(x, y, z) for x, y, z in zip(a, b, c)]
214
215 def find(cond, xs):
216     """First match, lazily, into Maybe.
217     >>> find(even, [1, 3, 4, 5])
218     4
219     >>> find(even, [1, 3]) is NOTHING
220     True
221     """
222     return next((x for x in xs if cond(x)), NOTHING)
223
224 def partition(cond, xs):
225     """(keepers, rest) in ONE pass, cond called once per element.
226     >>> partition(even, [1, 2, 3, 4])
227     ([2, 4], [1, 3])
228     """
229     yes, no = [], []
230     for x in xs:
231         (yes if cond(x) else no).append(x)
232     return (yes, no)
233
234 # ===== 9. slicing & spans =====
235 def span(cond, xs):
236     """Split at first failure, one pass, one cond call per element.
237     >>> span(lambda x: x < 3, [1, 2, 3, 1])
238     ([1, 2], [3, 1])
239     """
240     xs = list(xs)
241     i = next((i for i, x in enumerate(xs) if not cond(x)), len(xs))
242     return (xs[:i], xs[i:])
243
244 break_ = lambda cond, xs: span(o(not_), cond), xs)
245
246 def takewhile(cond, xs):
247     # generator
248     for x in xs:
249         if not cond(x):
250             return
251         yield x
252
253 def dropwhile(cond, xs):
254     # generator
255     it = iter(xs)
256     for x in it:
257         if not cond(x):
258             yield x
259             break
260     yield from it
261
262 def takeuntil(cond, xs):
263     # takewhile that INCLUDES the first hit
264     for x in xs:
265         yield x
266         if cond(x):
267             return
268
269 takeWhile = lambda cond, xs: list(takewhile(cond, xs))
270 dropWhile = lambda cond, xs: list(dropwhile(cond, xs))
271 inits     = lambda xs: [xs[:i] for i in range(len(xs) + 1)]
272 tails    = lambda xs: [xs[i:] for i in range(len(xs) + 1)]
273
274 def chunksOf(n, xs):
275     """

```

```

273     >>> chunksOf(2, [1, 2, 3, 4, 5])
274     [[1, 2], [3, 4], [5]]
275     """
276     xs = list(xs)
277     return [xs[i:i + n] for i in range(0, len(xs), n)]
278
279 def windows(n, xs):
280     """Sliding windows of width n.
281     >>> windows(3, [1, 2, 3, 4])
282     [[1, 2, 3], [2, 3, 4]]
283     """
284     xs = list(xs)
285     return [xs[i:i + n] for i in range(len(xs) - n + 1)]
286
287 def groupBy(eq, xs):
288     """Haskell semantics: runs anchored on each run's FIRST element.
289     >>> groupBy(lambda a, b: a == b, "aabbba")
290     [['a', 'a'], ['b', 'b'], ['a']]
291     """
292     out = []
293     for x in xs:
294         if out and eq(out[-1][0], x):
295             out[-1].append(x)
296         else:
297             out.append([x])
298     return out
299
300 group = partial(groupBy, lambda a, b: a == b)
301
302 def transpose(rows):
303     """Ragged-safe like Data.List.
304     >>> transpose([[1, 2, 3], [4, 5]])
305     [[1, 4], [2, 5], [3]]
306     """
307     rows = [list(r) for r in rows]
308     out, i = [], 0
309     while any(i < len(r) for r in rows):
310         out.append([r[i] for r in rows if i < len(r)])
311         i += 1
312     return out
313
314 # ===== 10. sorting & searching =====
315 sortOn = lambda f, xs: sorted(xs, key=f)
316 sortBy = lambda cmp, xs: sorted(xs, key=cmp_to_key(cmp))
317 maxOn = lambda f, xs: max(xs, key=f)
318 minOn = lambda f, xs: min(xs, key=f)
319 merge = lambda a, b: list(_hmerge(a, b)) # stable merge of sorted lists
320
321 # ===== 11. strings =====
322 lines = str.splitlines
323 unlines = lambda ls: "".join(l + "\n" for l in ls)
324 words = str.split
325 unwords = " ".join
326
327 # ===== 12. containers ===== (Counter, defaultdict, deque, heappush/heappop are stdlib's)
328 def fromListWith(f, pairs):
329     """Data.Map: THE dict-building fold; f(new, old).
330     >>> fromListWith(add, [('a', 1), ('b', 2), ('a', 3)]) == {'a': 4, 'b': 2}
331     True
332     """
333     d = {}
334     for k, v in pairs:
335         d[k] = f(v, d[k]) if k in d else v
336     return d
337
338 def insertWith(f, k, v, d):
339     """Persistent insert (fresh dict); f(new, old).
340     >>> insertWith(add, 'a', 5, {'a': 1}) == {'a': 6}
341     True
342     """
343     return {**d, k: f(v, d[k]) if k in d else v}
344
345 def unionWith(f, a, b):

```

```

346     """Data.Map: merge two dicts, combining shared keys with f.
347     >>> unionWith(add, {'a': 1, 'b': 2}, {'b': 3, 'c': 4}) == {'a': 1, 'b': 5, 'c': 4}
348     True
349     """
350     return {k: f(a[k], b[k]) if k in a and k in b else (a[k] if k in a else b[k])
351             for k in a.keys() | b.keys()}
352
353 def getpath(t, ks, default=NOTHING):
354     """Read along a key path without autovivifying; Maybe out.
355     >>> getpath({'a': {'b': None}}, ['a', 'b']) is None
356     True
357     >>> getpath({'a': {}}, ['a', 'b']) is NOTHING
358     True
359     """
360     for k in ks:
361         if not isinstance(t, dict) or k not in t:
362             return default
363         t = t[k]
364     return t
365
366 # multiset ops on count-dicts (use .get -- indexing a Counter autovivifies):
367 bag_diff = lambda a, b: {k: a[k] - b.get(k, 0) for k in a if a[k] > b.get(k, 0)}
368 bag_inter = lambda a, b: {k: v for k in a.keys() & b.keys() if (v := min(a[k], b[k]))}
369 bag_union = lambda a, b: {k: max(a.get(k, 0), b.get(k, 0)) for k in a.keys() | b.keys()}
370 bag_sub = lambda a, b: all(b.get(k, 0) >= n for k, n in a.items())
371
372 # ===== 13. Maybe ===== (NOTHING sentinel; None is a legal value)
373 def bind(x, f):
374     """>= : short-circuit on NOTHING.
375     >>> bind(3, lambda v: v + 1), bind(NOTHING, lambda v: v + 1)
376     (4, NOTHING)
377     """
378     return NOTHING if x is NOTHING else f(x)
379
380 isJust = lambda x: x is not NOTHING
381 isNothing = lambda x: x is NOTHING
382 fromMaybe = lambda default, x: default if x is NOTHING else x
383
384 listToMaybe = lambda xs: xs[0] if xs else NOTHING
385 maybeToList = lambda x: [] if x is NOTHING else [x]
386
387 def catMaybes(xs):
388     """
389     >>> catMaybes([1, NOTHING, None, 2])
390     [1, None, 2]
391     """
392     return [x for x in xs if x is not NOTHING]
393
394 def mapMaybe(f, xs):
395     """Map into Maybe and drop the NOTHINGs, one pass.
396     >>> mapMaybe(lambda s: int(s) if s.isdigit() else NOTHING, ['3', 'x', '7'])
397     [3, 7]
398     """
399     return [y for y in map(f, xs) if y is not NOTHING]
400
401 def sequenceM(xs):
402     """All-or-nothing.
403     >>> sequenceM([1, 2]), sequenceM([1, NOTHING])
404     ([1, 2], NOTHING)
405     """
406     out = []
407     for x in xs:
408         if x is NOTHING:
409             return NOTHING
410         out.append(x)
411     return out
412
413 traverseM = lambda f, xs: sequenceM(map_(f, xs))
414
415 def maybe_get(d, k):
416     """dict lookup into Maybe (None stays distinguishable from missing).
417     >>> maybe_get({'a': None}, 'a'), maybe_get({}, 'a')
418     (None, NOTHING)

```

```

419     """
420     return d.get(k, NOTHING)
421
422 # ===== 14. Either ===== (tagged tuples; match-friendly)
423 Ok = lambda v: ("ok", v)
424 Err = lambda e: ("err", e)
425
426 def bindE(x, f):
427     """
428     >>> bindE(Ok(3), lambda v: Ok(v + 1)), bindE(Err("no"), lambda v: Ok(v))
429     (('ok', 4), ('err', 'no'))
430     """
431     return x if x[0] == "err" else f(x[1])
432
433 def sequenceE(xs):
434     """First Err wins; else Ok of all values.
435     >>> sequenceE([Ok(1), Err("a"), Err("b")])
436     ('err', 'a')
437     """
438     out = []
439     for x in xs:
440         if x[0] == "err":
441             return x
442         out.append(x[1])
443     return Ok(out)
444
445 def traverseE(f, xs):
446     """Map into Either, then sequence.
447     >>> traverseE(lambda n: Ok(n * 2) if n else Err('zero'), [1, 2])
448     ('ok', [2, 4])
449     """
450     return sequenceE(map_(f, xs))
451
452 def partitionEithers(xs):
453     """
454     >>> partitionEithers([Ok(1), Err('a'), Ok(2)])
455     ([1, 2], ['a'])
456     """
457     oks, errs = [], []
458     for tag, v in xs:
459         (oks if tag == "ok" else errs).append(v)
460     return (oks, errs)
461
462 def note(msg, x):
463     """Maybe -> Either: attach WHY to a NOTHING.
464     >>> note("missing key", NOTHING)
465     ('err', 'missing key')
466     """
467     return Err(msg) if x is NOTHING else Ok(x)
468
469 # ===== 15. do-notation ===== (yield is >>=; linear monads only)
470 def do(binder, is_fail):
471     """Build a do-block decorator for any short-circuiting monad."""
472     def deco(gen_fn):
473         def run(*a, **kw):
474             g = gen_fn(*a, **kw)
475             def step(val):
476                 try:
477                     m = g.send(val)
478                 except StopIteration as e:
479                     return e.value
480                 return binder(m, step)
481             return step(None)
482         return run
483     return deco
484
485 doM = do(bind, lambda x: x is NOTHING) # generator returns a plain value
486 doE = do(bindE, lambda x: x[0] == "err") # generator returns Ok(...)/Err(...)
487
488 # ===== 16. parser combinators =====
489 # Parser a = str -> (value, rest) | FAIL. Values may be None (json null!).
490 def doP(gen_fn):
491     """do-notation that THREADS the input: yield a parser, receive its value.

```

```

492 Fresh generator per parse -- re-runnable, backtrack-safe under alt. """
493 def make(*a, **kw):
494     def parser(s):
495         g, val = gen_fn(*a, **kw), None
496         try:
497             while True:
498                 r = g.send(val)(s)
499                 if r is FAIL:
500                     return FAIL
501                 val, s = r
502             except StopIteration as e:
503                 return (e.value, s)
504         return parser
505     return make
506
507 def alt(*ps):
508     """</> : first success wins. """
509     return lambda s: next((r for r in (p(s) for p in ps) if r is not FAIL), FAIL)
510
511 def many(p):
512     """Zero or more, loop not recursion; stalls guard against empty matches. """
513     def parser(s):
514         out = []
515         while (r := p(s)) is not FAIL and r[1] != s:
516             out.append(r[0]); s = r[1]
517         return (out, s)
518     return parser
519
520 def many1(p):
521     def parser(s):
522         r = p(s)
523         if r is FAIL:
524             return FAIL
525         rest = many(p)(r[1])
526         return ([r[0]] + rest[0], rest[1])
527     return parser
528
529 def sepBy(p, sep):
530     def pair(s):
531         r = sep(s)
532         return FAIL if r is FAIL else p(r[1])
533     def parser(s):
534         r = p(s)
535         if r is FAIL:
536             return ([], s)
537         rest = many(pair)(r[1])
538         return ([r[0]] + rest[0], rest[1])
539     return parser
540
541 def lit(c):
542     """One expected character, whitespace-blind. """
543     def parser(s):
544         s = s.lstrip()
545         return (c, s[1:]) if s[:1] == c else FAIL
546     return parser
547
548 def rx(pat, conv=identity):
549     """Regex token, whitespace-blind, converted. """
550     r = _re.compile(pat)
551     def parser(s):
552         s = s.lstrip()
553         m = r.match(s)
554         return (conv(m.group()), s[m.end():]) if m else FAIL
555     return parser
556
557 def chainl1(p, ops):
558     """p (op p)* folded LEFT; ops maps operator char -> binary function. """
559     opp = alt(*[lit(c) for c in ops])
560     def parser(s):
561         vr = p(s)
562         while vr is not FAIL:
563             op = opp(vr[1])
564             if op is FAIL:

```

```

565         return vr
566     w = p(op[1])
567     if w is FAIL:
568         return vr
569     vr = (ops[op[0]](vr[0], w[0]), w[1])
570     return FAIL
571 return parser
572
573 def runParser(p, s):
574     """Parser -> Either: Ok(value) or Err showing where it stopped.
575     >>> runParser(rx(r'\d+', int), "42")
576     ('ok', 42)
577     >>> runParser(rx(r'\d+', int), "42x")
578     ('err', "unconsumed input: 'x'")
579     """
580     r = p(s)
581     if r is FAIL:
582         return Err(f"no parse: {s!r}")
583     if r[1].strip():
584         return Err(f"unconsumed input: {r[1].strip()!r}")
585     return Ok(r[0])
586
587 # ===== 17. monoids =====
588 Monoid = lambda empty, op: (empty, op)
589 mconcat = lambda m, xs: foldl(m[1], xs, m[0])
590 foldMap = lambda f, m, xs: mconcat(m, [f(x) for x in xs])
591 both = lambda m1, m2: Monoid((m1[0], m2[0]),
592                               lambda a, b: (m1[1](a[0], b[0]), m2[1](a[1], b[1]))))
593 Sum = Monoid(0, add)
594 Product = Monoid(1, mul)
595 All = Monoid(True, lambda a, b: a and b)
596 Any = Monoid(False, lambda a, b: a or b)
597 MaxM = Monoid(float("-inf"), max)
598 MinM = Monoid(float("inf"), min)
599 ListM = Monoid([], add)
600 First = Monoid(NOTHING, lambda a, b: b if a is NOTHING else a)
601 Last = Monoid(NOTHING, lambda a, b: a if b is NOTHING else b)
602
603 if __name__ == "__main__":
604     import doctest
605     fails, total = doctest.testmod()
606     print(f"{total - fails}/{total} doctests passed" if not fails
607           else f"{fails} FAILED")

```

